Geometric and Learned Relay Control for Connectivity-Constrained Drone Delivery in Disaster Response

Gwang-Jong Ko[a], Taesu Cheong[a,*]

[a]*School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul, 02841, Republic of Korea*

Abstract

In disaster relief, delivery drones must stay connected to a depot through a multi-hop relay chain formed by the fleet itself, while carrying supplies with limited capacity and returning to reload. We model this as a makespan-minimisation problem in which every drone can either deliver or relay, and connectivity is a hard constraint enforced at every time step. We propose two controllers that share the same connectivity-preserving projection: a geometric chain-relay rule with a deadlock-recovery mechanism, and a fleet-size-agnostic policy learned once with discrete soft actor-critic over a set-based, autoregressive manager. Both are compared on identical instances with a paired protocol across fleets of 3 to 12 drones and 20 to 400 demand points, against four learning-free baselines and a time-expanded mixed-integer reference model on small instances. The learned policy completes every instance in every configuration; the rule completes 98–100% with the recovery mechanism and 62% without it. The two controllers divide the configuration space: with three drones the rule is 30–39 steps faster in paired median, with four drones the two are statistically indistinguishable over 480 paired runs, and with five or more drones the learned policy is faster by 38 to 178 steps. The policy is trained only on three and four drones and transfers without retraining to every larger fleet tested; the set-based architecture carries completion and training on randomised configurations carries speed. The three-drone deficit is localised to a single decision, which drone relays, and eleven policy-side interventions covering information, credit assignment, decision structure, initialisation and reward shaping all fail to remove it; half of it disappears when the connectivity projection is changed from radial shrinking to tangential sliding, showing that the deficit is partly a property of how the constraint is enforced rather than of the learner. On forty instances solved to proven optimality, with the controllers' relays restricted to the model's own node set, the learned policy is 4.3% above the optimum on average and the rule 9.7%. Both controllers run in milliseconds per decision, three to six orders of magnitude below exact optimisation, which no longer proves optimality once reloading is needed.

Keywords: drone delivery, relay network, connectivity constraint, multi-agent reinforcement learning, rule, disaster logistics

1. Introduction

When communication, logistics and road infrastructure are destroyed, a fleet of drones can carry emergency supplies from a temporary depot to many demand points. Drones have been studied for last-mile distribution in disaster relief [1, 2, 3] and for medical delivery [4, 5], and the routing of drone fleets has grown into a literature of its own since the flying-sidekick travelling salesman problem [6, 7, 8, 9]. Two features make the disaster setting different from parcel delivery. First, every drone must be reachable by the depot at all times, because loading and unloading are supervised remotely and the delivery status must be tracked; with the ground network gone, reachability is provided only by the drones themselves acting as radio relays in a multi-hop chain. Second, the fleet is small and each drone carries a few units, so drones shuttle between the depot and the demand points and every drone that stands still as a relay is a drone that is not delivering. The operational question is how to assign delivery and relay roles over time so that the last demand point is served as early as possible without ever breaking the chain. Aerial relays are well studied in the communications literature [10, 11, 12], and connectivity maintenance is a classical topic in multi-robot control [13, 14], but in both the relay *is* the mission; here relaying competes with the delivery mission for the same vehicles.

Three properties make the problem hard. A hard connectivity constraint creates deadlocks: a greedy assignment that sends every drone to its nearest demand point completes none of the standard instances in our study, because the fleet pulls itself apart until no drone can move. Relaying competes directly with throughput, so the right number and identity of relays changes with the geometry of the remaining demand points and with how many drones are left with cargo. And exact optimisation does not scale: a space–time network model [15, 16] on a 98-metre lattice is solved to proven optimality in seconds while no reloading is needed, but once drones must reload the mixed-integer solver proves optimality on few instances within an hour and returns no feasible solution on some, whereas drone-routing models without a connectivity constraint are solved exactly for tens of customers [17].

*Corresponding author.

Email addresses: koptimizer@korea.ac.kr (Gwang-Jong Ko), tcheong@korea.ac.kr (Taesu Cheong)

Proposal. We propose three methods for this problem and study them side by side: two online controllers that decide in milliseconds, and one offline search that we use as a quality reference.

1. A *geometric chain-relay rule*. The drone farthest from the depot with cargo is the lead; the number of relays is the number of communication hops its target needs; the relays are the next-farthest drones, stationed on the segment from the depot to the lead at intervals of $0.9R$. Because the rule is deterministic it can repeat a deadlocked assignment indefinitely, so we add a *deadlock-recovery* mechanism that swaps the lead whenever the fleet has stalled for a fixed number of steps. The recovery mechanism alone lifts completion on the standard configuration from 62% to 98%.
2. A *search heuristic* that keeps the rule's option structure and its relay geometry but replaces the myopic target choice by simulated annealing over per-drone visit orders, evaluated by simulation. It is offline and costs minutes per instance, and we use it to separate how much of the distance to the optimum is the option structure and how much is the ordering.
3. A *learned policy* that is agnostic to fleet size. A manager assigns each drone one of seven discrete options every twenty steps; it encodes the drones, candidate demand points and relay slot as a set, pools them with attention, and decides autoregressively so that a formation of one lead and k relays can be composed deliberately. It is trained once with discrete soft actor-critic [18] on fleets of three and four drones and then run without retraining on fleets of up to twelve, in the spirit of size-agnostic learned solvers for routing [19, 20].

All three act through the same connectivity-preserving projection: each commanded move is scaled by the largest factor that keeps the fleet connected. Connectivity is therefore guaranteed by construction for both, and the empirical question is whether a controller makes progress under the constraint rather than whether it violates it.

Research questions. We organise the study around five questions.

RQ1 Do the proposed controllers always complete the delivery while connectivity is enforced at every step, and when they do not, why not?

RQ2 On small instances, how close are the proposed controllers to the optimal solution of a space–time network model solved exactly?

RQ3 How does the performance of the two controllers compare across operating conditions, in particular across fleet size, number of demand points, depot position and communication radius, and where do the differences come from?

RQ4 Does the learned policy, trained on fleets of three and four drones, transfer upward to larger fleets and larger instances without retraining, and what carries the transfer?

RQ5 Are the components of each controller necessary?

Findings. The learned policy completes every instance in every configuration tested; the rule completes 98–100% with the recovery mechanism (RQ1). On forty instances solved to proven optimality, with both controllers' relays restricted to the model's node set, the learned policy is 4.3% above the optimum on average and the rule 9.7%, and the optimum is itself an upper bound on the continuous optimum (RQ2). The two controllers divide the configuration space by fleet size: the rule is faster with three drones, the two are statistically indistinguishable with four, and the learned policy is faster with five or more, by 62 to 178 steps up to ten drones and 76 at twelve; the three-drone deficit is a single decision, which drone relays, and it resists eleven policy-side interventions but shrinks by half when the connectivity projection is changed (RQ3). The learned policy never saw more than four drones or 80 demand points in training, yet it completes every instance with up to twelve drones and 400 demand points; the set-based architecture carries completion and the configuration randomisation in training carries speed (RQ4). Removing rule regularisation, configuration randomisation or entropy annealing from the learned policy costs hundreds of steps or completion, and removing the recovery mechanism from the rule costs a third of its completions (RQ5). The learned policy's heavy right tail resists training-side remedies but is removed by applying the rule's recovery step at inference.

Contributions. (i) A geometric chain-relay rule with deadlock recovery and a fleet-size-agnostic learned policy for connectivity-constrained delivery, both built on a projection that makes the constraint hold by construction. (ii) A paired evaluation across fleet sizes that shows the two controllers are complementary, together with the mechanism behind the boundary, and evidence that the learned policy transfers upward to fleets three times larger than any it was trained on. (iii) A set of eleven documented negative results on closing the relay-limited deficit by changing the learner, and a sensitivity result showing that half of the deficit is attributable to the enforcement model. (iv) A space–time network model of the problem, solved exactly on small instances, with an explicit account of what its optimum does and does not bound.

2. Related Work

2.1. Vehicle routing with drones

The problem belongs to the family of vehicle routing problems [21, 22], with a homogeneous capacitated fleet, a single depot, unit demands and a makespan objective; the makespan (or min-max) objective, as opposed to total distance, is the standard choice when the goal is to finish service as early as possible [20, 23]. Drone routing began with the flying-sidekick travelling salesman problem, in which a truck and a drone cooperate [6], and was extended to multiple drones [24], to exact and approximate solution methods [25, 26, 17, 27], to vehicle routing with drones [28, 29, 30] and to drone-only fleets with energy and payload constraints [31]. The surveys of Otto et al. [7], Macrina et al. [8] and Chung et al. [9] map this literature.

Across all of it communication is free: a drone is never required to be reachable from the depot, and no vehicle gives up its delivery role to serve as a relay. Our problem adds exactly that requirement, and the main consequence for modelling is that routes are no longer sufficient as decision variables, because the constraint concerns where all vehicles are at every instant (Section 3.1). Dynamic vehicle routing [32] shares the need to decide online as the state evolves, which is the mode in which both of our controllers operate.

2.2. Drones in disaster and humanitarian logistics

Drones have been proposed for last-mile distribution when ground infrastructure is damaged. Chowdhury et al. [1] design a drone-based relief distribution network with a continuous approximation model; Rabta et al. [2] optimise a drone fleet for last-mile delivery under payload and range limits; Kim et al. [4] and Ghelichi et al. [5] plan drone fleets for medical deliveries in rural and urban settings; and Erdelj et al. [3] survey the roles drones can take in disaster management, including that of temporary communication nodes. These works plan at the strategic or tactical level (fleet size, depot location, route assignment) and assume that the control link exists. Our contribution is at the operational level: given a fleet and a depot, how the drones should move so that the link is maintained by the fleet itself while it delivers.

2.3. Aerial relays and connectivity maintenance

Using UAVs as communication relays is a mature topic in wireless networking: surveys cover flying ad-hoc networks [33], UAV communication issues [34] and UAV-enabled wireless networks [11]; throughput-optimal relay trajectories are derived in [10]; and relay placement for maintaining a link to a surveillance UAV is solved exactly in [12]. Multi-hop chains of mobile relays are studied in the robotics literature as communication chains [14], robotic routers [35] and relay positioning for team missions [36]. Connectivity maintenance for a team of mobile robots has a control-theoretic literature based on the algebraic connectivity of the communication graph [37, 13], and learned connectivity-preserving controllers exist [38]. Learning-based relay control has been applied to packet routing in UAV networks [39], to joint trajectory and power control of relay and mission UAVs [40] and to aerial relay networking [41]. In all of these the relays are the mission, or a dedicated sub-fleet with its own reward; they do not carry cargo, do not reload and do not minimise a delivery makespan. Switching between a task role and a relay role is an established idea in multi-robot exploration [42, 43], where the relay robots land and act as fixed nodes; our drones switch roles repeatedly and the relay positions move with the lead drone.

2.4. Time-expanded network models and symmetry

Our exact model is a time-indexed formulation on a space–time network. Time-expanded networks originate with Ford and Fulkerson's flows over time [15]; Skutella [16] surveys the theory. The dependence of such models on the time discretisation is well known, and dynamic discretisation discovery [44] refines the time grid where needed; we discretise space as well, which is the usual device in MILP trajectory planning for aircraft [45]. Connectivity is encoded as a single-commodity flow per period whose feasibility is equivalent to the existence of a multi-hop chain (Section 3.1). The drones are identical, so the model carries the usual index symmetry, which we break with the ordering constraints of Sherali and Smith [46]; Margot [47] surveys symmetry handling in integer programming.

2.5. Learning for routing and cooperative control

Neural solvers for routing problems, from pointer networks [48] to attention-based constructive policies [49, 19], show that set-based encoders with attention generalise across instance sizes; Bengio et al. [50] and Mazyavkina et al. [51] survey machine learning for combinatorial optimisation. Learned solvers for the min-max multiple travelling salesman problem [20, 52] are the closest in objective to ours, and we borrow the size-agnostic set encoding; none of them has a connectivity constraint, and none decides online. Dual-mission fleets in which the same drones deliver and sense [53] formalise a per-delivery detour budget, but the secondary task there is sensing rather than communication and it does not couple the vehicles' positions. On the learning side we build on soft actor-critic [18], centralised critics for decentralised execution [54, 55], value decomposition [56, 57], cooperative multi-agent policy optimisation [58] (see [59] for a review), temporal abstraction through options [60, 61, 62], invalid-action masking [63] and safe-action projection [64]. Our negative results in Section 6.3 test several of these ideas directly: per-agent credit, joint-action critics, monotonic value mixing and potential-style shaping [65] are all tried as remedies for the relay-limited deficit and none of them removes it.

2.6. Position of this work

To the best of our knowledge, no prior work combines (i) a homogeneous fleet carrying physical cargo to many demand points with capacity and reloading, (ii) a hard requirement that every drone be connected to the depot by a multi-hop chain at every step, and (iii) a makespan objective. In this setting relay duty competes with cargo throughput and the fleet must return to the depot repeatedly, which is what makes role assignment a sequential decision rather than a one-off placement. We contribute an exact space–time network model of this problem, a constructive online rule with a deadlock-recovery mechanism, and a learned size-agnostic policy, and we report where each is preferable.

3. Problem Statement

A depot sits at $c \in [0, 1000]^2$ metres and M demand points $d_1, \ldots, d_M$ lie within $0.9\,NR$ of it, where N is the number of drones and $R = 300\,\text{m}$ the communication range. Each drone flies at $v = 13.89\,\text{m/s}$ and carries $Q = 5$ units. Every delivery and every reload incurs a *service time*: unloading one unit at a demand point takes $S = 10$ steps and reloading to Q at the depot takes S steps, and during these steps the drone is stationary. A

demand point or the depot is reached when a drone is within 25 m of it, and one step is one second. The service times matter for the connectivity constraint below: a stationary drone cannot move to keep a link alive, so its teammates must keep the chain intact around it. Every demand point needs exactly one unit. Drones do not collide, are not affected by wind, and battery is not modelled.

Connectivity. Let G_t be the graph whose vertices are the depot and the N drones and whose edges join vertices within R of each other. The fleet is *connected* at step t if G_t is connected, which is the same as saying that every drone has a multi-hop path to the depot. The constraint is that G_t is connected at every t. It is enforced rather than penalised (Section 4.1).

Objective. The makespan is the step at which the last demand point is served. A controller *completes* an instance if it serves all demand points within a budget of 10,000 steps. We report completion rate and, over completed instances, makespan.

Configurations. A configuration is a triple of fleet size N, number of demand points M and depot placement (fixed at the map centre, or uniformly random). The *standard* configuration is $N = 4$, $M = 50$ with the fixed depot. The learned policy is trained with $N \in \{3, 4\}$, $M \in [20, 80]$ and a random depot; all other configurations are unseen by it. The rule has no training.

3.1. A space–time network model, exact on the discretisation

The third element we propose, besides the two controllers, is an optimisation model of the problem. It serves two purposes: it states the problem without ambiguity, and on small instances it provides a plan obtained by exact optimisation against which both controllers are measured.

Why the model takes this form. Vehicle-routing formulations of drone delivery [6, 25] assign routes to vehicles and never need to know where a vehicle is at a given instant. The connectivity requirement of Section 3 is different in kind: it is a constraint on the *positions of all drones at every step*, because the communication graph G_t is built from those positions. A model must therefore carry every drone's position over time, and the natural object for that is a *time-expanded network* [15]: the set of positions is copied once per time period and arcs join a position in one period to the positions reachable in the next. Positions in the plane are uncountable, so space is discretised as well, to a lattice; the result is a *space–time network* on which drones are unit flows, and a time-indexed MILP on this network is exact for the discretised problem. Connectivity is then a property of which lattice nodes are occupied in each period, and we encode it with a single-commodity flow per period that is feasible exactly when the occupied nodes form a multi-hop chain to the depot. The model is *exact* in the sense that its optimum is the true optimum of the discretised problem; its relation to the continuous problem is stated at the end of this section.

Discretisation. A period is $\kappa = S = 10$ steps, so a service occupies exactly one period and a manager decision of the learned policy spans two; the per-period flight range is $\ell = v\kappa = 138.9\,\text{m}$. Space is a square lattice G of spacing $h = \ell/\sqrt{2} = 98.2\,\text{m}$ over the bounding box of the instance, from which nodes farther than $0.9NR$ from the depot are deleted because no chain can reach them; the depot and every demand point are added as exact nodes, so the interaction test is exact there. Figure 1 shows the resulting node set for a reference instance. With this spacing all eight king moves are feasible in one period; the move set and the communication set are

$$A(i) = \{j \in N_{\text{v}} : \|p_i - p_j\| \le \ell\} \ni i, \tag{1}$$

$$E = \{(i, j) \in N_{\text{v}}^2 : i \ne j,\ \|p_i - p_j\| \le R\}. \tag{2}$$

Axis-aligned moves cover h rather than ℓ per period, so the lattice slows a drone by up to 29% in those directions; this discretisation error always acts against the model. The lattice must also be fine enough to hold a relay chain: serving a demand point at distance d needs the intermediate drones placed within the chain's slack $NR - d$, which requires roughly $h \lesssim (NR - d_{\max})/(N - 1)$; a 196 m lattice makes the reference instances infeasible although they are feasible in the plane, and 98 m does not.

Node pruning by the convex hull. Most lattice nodes inside the reach limit are never useful. Let H be the convex hull of the depot and the demand points. For any configuration of the fleet, projecting every drone that lies outside H onto H does not increase any pairwise distance, because projection onto a convex set is non-expansive; hence it preserves every communication link, every one-period move and every service, and the projected plan has the same makespan. The continuous problem therefore always has an optimal plan inside H, and relay positions outside the hull can be discarded without loss. On the lattice we keep the nodes within $h/\sqrt{2}$ of H, the half-diagonal of a cell, so that every point of H still has a kept node within one lattice step; this margin guards against the case in which the projection of a lattice node is not itself a lattice node. The pruning removes 45% of the nodes of the 34 reference instances on average (from 2,881 to 1,594 in total) and, on the instances where we can check, leaves the lattice optimum unchanged (Section 6.2). Figure 1 shows the pruned and kept nodes.

Notation. Table 1 lists the sets, parameters and decision variables. Drones are identical, so a drone index only distinguishes units; the symmetry-breaking cut at the end of the model exploits this. The model is given in (3)–(16).

Reading the model. The objective (3) minimises the makespan C; the second term is a tie-break with weight $\varepsilon\bar{T}N \ll 1$ that prefers plans keeping idle drones at the depot and cannot change the optimal makespan.

Position and movement, (4)–(6). Every drone is at exactly one node at every period boundary and starts at the depot. A drone may be at node i at boundary $t + 1$ only if at boundary t it was at i itself or at a node from which i is reachable within

Table 1: Notation of the space–time network model.

Symbol	Domain	Meaning
Sets		
$N_v = \{0\} \cup D \cup G$		nodes: the depot (index 0), the demand-point nodes D, the lattice nodes G
K	$\lvert K\rvert = N$	drones
$T = \{0, 1, \ldots, \bar{T}\}$		period boundaries; period t runs from boundary t to $t + 1$
$A(i) \subseteq N_v$		nodes reachable from i within one period, including i (move set)
$E \subseteq N_v^2$		ordered pairs of distinct nodes within communication range R (communication arcs)
Parameters		
Q	5	cargo capacity (units)
R	300 m	communication range
κ, ℓ, h	10 steps, 138.9 m, 98.2 m	period length, flight range per period, lattice spacing
$\bar{T}$		horizon in periods, set to 1.15× the rule's makespan
ε	10^{-5}	tie-breaking weight
idx(j)		index of demand point j in a fixed ordering of D
Decision variables		
y_{ikt}	$\{0, 1\}$	1 if drone k is at node i at boundary t
s_{jkt}	$\{0, 1\}$	1 if drone k delivers to demand point j during period t
r_{kt}	$\{0, 1\}$	1 if drone k reloads at the depot during period t
q_{kt}	$\{0, \ldots, Q\}$	cargo on board drone k at boundary t
ρ_{kt}	$[0, Q]$	cargo taken on by drone k during period t
g_{ijt}	$[0, N]$	connectivity flow on arc (i, j) in period t
u_{it}	$\{0, \ldots, N\}$	occupancy of node i at boundary t, $u_{it} = \sum_k y_{ikt}$, with $u_{0t} := 1$
C	$[0, \bar{T}]$	makespan in periods

one period. Because every move takes exactly one period, the position of every drone is defined at every boundary, which is what the connectivity constraints below need.

Deliveries, cargo and reloading, (7)–(12). Each demand point is served exactly once (7). A delivery during period t requires the drone to be at the demand point at both ends of the period (8), which is the service time of Section 3: the drone stands still for one period. Reloading is likewise a full period at the depot (9). A drone does at most one service per period and can only deliver what it carries (10); cargo is tracked by (11) starting full; and a reload fills the drone completely (12), because ρ_{kt} can be positive only in a reload period and the next cargo must then be Q.

Multi-hop connectivity, (13)–(14). These two families are the part that distinguishes this model from drone-routing models. In every period, every occupied node ships one unit of flow, the depot is the only sink, and an arc may carry flow only if it leaves an occupied node. The system is feasible if and only if every occupied node is joined to the depot by a path whose intermediate nodes are occupied and whose consecutive distances are at most R, that is, if and only if the simulator's multi-hop connectivity test succeeds at that period boundary. The proof is short: if such paths exist, routing each node's unit along its path satisfies both families; conversely an unoccupied node $i \neq 0$ has $g_{ijt} = 0$ on all its outgoing arcs by (14) and therefore no incoming flow by (13), so all flow travels inside the occupied set and, since the depot is the only node without a conservation row, a flow decomposition yields the required paths. The mirror-image capacity rows $g_{ijt} \leq N u_{jt}$ are implied and omitted, which halves the dominant row count. Note what the constraint does to the problem: a drone that has nothing to deliver cannot simply wait at the centre if a teammate needs it as a hop, and a drone cannot fly to a demand point that would leave it farther than R from every occupied node. Relaying is therefore not a separate task but a consequence of where the fleet is.

Makespan and symmetry, (15)–(16). The makespan is at least the period of every delivery. Because drones are identical, any plan can be relabelled so that drone k serves no demand point with index below k; (16) imposes this ordering and removes no objective value.

What the optimum means. The optimum of the model is exact for the discretised problem. For the continuous problem of Section 3 it is an *upper bound*: drones are confined to lattice nodes and axis moves are shortened, so every lattice plan is a feasible continuous plan but not conversely, and a continuous controller can therefore be faster than the lattice optimum. The solver's dual bound is a bound on the lattice problem only. A certified lower bound through a cell relaxation, in which a lattice node stands for its cell and communication is tested by the minimum distance between cells, was attempted and abandoned: it inflates a three-hop chain's reach from 900 m to 1,317 m and returns bounds below the trivial analytic bound. We therefore call the solver's value a *reference plan*, replay every plan in the continuous simulator, and report the replayed makespan.

4. Proposed Controllers

Figure 2 summarises the setting and the two controllers. Both controllers decide at the level of *goals*: at every decision point

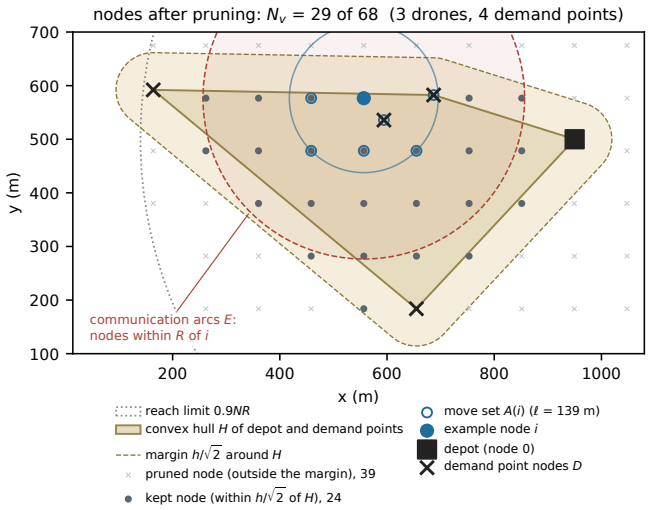


Figure 1: Nodes of the space–time network for a reference instance with three drones and four demand points: the depot (node 0), the demand-point nodes D, and the lattice G of spacing $h = 98.2$ m clipped to the reach limit $0.9NR$ (dotted). The shaded polygon is the convex hull H of the depot and the demand points and the lighter band is the margin of width $h/\sqrt{2}$ around it; lattice nodes inside the band or the hull are kept (dots) and those outside are pruned (crosses), leaving 29 of 68 nodes. For one example node i the figure marks its move set $A(i)$ (the nodes reachable in one period, $\ell = 138.9$ m) and its communication disc of radius R, which defines its arcs in E.

each drone is given a goal (a demand point, the depot, or a relay slot) and flies towards it in a straight line at full speed until the next decision point. The two controllers differ only in how goals are chosen; the option set, the low-level flight, the relay-slot geometry and the connectivity projection are shared and are described first.

4.1. Shared interface: options, relay slots, projection

Decision points and options. A decision point occurs every 20 steps, and additionally as soon as any drone's goal becomes invalid: its demand point has been served by another drone, it has run out of cargo while heading to a demand point, it has become full while heading to the depot, or its relay slot has disappeared because the chain it belonged to shortened. At a decision point every drone receives one of $K = 7$ options: one of its five *candidate demand points*, which are the five unserved demand points nearest to it; *return* to the depot; or *relay*. An option is *valid* for a drone only if it makes sense at that moment: a demand point option requires cargo on board and an existing candidate, return requires the drone not to be full, and relay requires a free relay slot (defined next) and at least one teammate that is not itself relaying. Invalid options are masked for the learned policy and never produced by the rule; a drone with no valid option returns to the depot.

Relay slots. The environment, not the controller, decides where a relaying drone stands. For drone i the *frontier* f_i is the point farthest from the depot among the delivery goals of the other drones and the current positions of the other non-relaying drones (if there is none, the farthest unserved demand point). A chain to f_i at distance d needs $n = \min(\lceil d/(0.9R) \rceil - 1,\ N - 1)$ relay slots, placed on the segment from the depot to f_i at the fractions $1/(n+1), \ldots, n/(n+1)$ of its length; the factor 0.9 leaves slack so that a drone standing slightly off its slot does not break the link. Drone i takes the free slot nearest to it. A drone whose slot disappears holds position until the next decision point. The controllers therefore decide only *whether* a drone relays; the slot follows from the fleet's geometry.

Low-level flight and connectivity projection. Every step, each drone commands a displacement of length v towards its goal (zero while it is serving). Enforcing connectivity by a reward penalty would punish situations that no action can prevent, for example a drone that is stationary while unloading and is stranded when a teammate departs; connectivity is therefore enforced by projection. If the commanded positions keep G_t connected they are applied. Otherwise the projection repeatedly selects one moving drone, scales its displacement down to the largest factor in $[0, 1]$ under which the fleet is connected (found by bisection with the other drones' current scalings fixed), and stops as soon as the fleet is connected; at each round it selects the drone whose restriction leaves the most drones connected and breaks ties at random, so that no single anchor drone is starved. The previous positions are always connected, so zero scaling is a valid fallback and the constraint holds by construction. A drone whose displacement is scaled below 10% is *stalled* for that step, and a *partial deadlock* is a step in which all but at most one of the moving drones are stalled; the number of consecutive partial-deadlock steps, δ_t, is what the rule's recovery mechanism monitors. This projection scales a displacement along its own direction; Section 6.3 studies a variant that keeps the tangential component.

4.2. Geometric chain-relay rule with deadlock recovery

The rule is a deterministic rule evaluated at every decision point (Procedure 1). It first gives every drone a default: a drone with cargo goes to its nearest unserved demand point and a drone without cargo returns to the depot. It then selects the *lead*, the drone with cargo that is farthest from the depot, computes how many hops the lead's target needs, and turns that many of the remaining drones, the farthest ones first, into relays. The number of relays is adaptive and the relays are the drones already closest to where the chain must stand, which is what makes the rule cheap: it costs $O(N \log N)$ per decision and 0.05–0.12 ms in our implementation.

Deadlock recovery. Because the rule is deterministic, a geometry that stalls the fleet makes it re-issue the same assignment at every decision point and the stall persists; without an recovery

$$\min \; C + \varepsilon \sum_{i \in N_v \setminus \{0\}} \sum_{k \in K} \sum_{t \in T} y_{ikt} \tag{3}$$

$$\text{s.t.} \; \sum_{i \in N_v} y_{ikt} = 1 \qquad \forall k \in K,\ t \in T \tag{4}$$

$$y_{0k0} = 1 \qquad \forall k \in K \tag{5}$$

$$y_{i,k,t+1} \le \sum_{j \in A(i)} y_{jkt} \qquad \forall i \in N_v,\ k \in K,\ t < \bar{T} \tag{6}$$

$$\sum_{k \in K} \sum_{t < \bar{T}} s_{jkt} = 1 \qquad \forall j \in D \tag{7}$$

$$s_{jkt} \le y_{jkt}, \qquad s_{jkt} \le y_{j,k,t+1} \qquad \forall j \in D,\ k \in K,\ t < \bar{T} \tag{8}$$

$$r_{kt} \le y_{0kt}, \qquad r_{kt} \le y_{0,k,t+1} \qquad \forall k \in K,\ t < \bar{T} \tag{9}$$

$$\sum_{j \in D} s_{jkt} + r_{kt} \le 1, \qquad \sum_{j \in D} s_{jkt} \le q_{kt} \qquad \forall k \in K,\ t < \bar{T} \tag{10}$$

$$q_{k,t+1} = q_{kt} - \sum_{j \in D} s_{jkt} + \rho_{kt}, \qquad q_{k0} = Q \qquad \forall k \in K,\ t < \bar{T} \tag{11}$$

$$\rho_{kt} \le Q\, r_{kt}, \qquad q_{k,t+1} \ge Q\, r_{kt} \qquad \forall k \in K,\ t < \bar{T} \tag{12}$$

$$\sum_{j:(i,j) \in E} g_{ijt} \; - \sum_{j:(j,i) \in E} g_{jit} = u_{it} \qquad \forall i \in N_v \setminus \{0\},\ t \in T \tag{13}$$

$$g_{ijt} \le N\, u_{it} \qquad \forall (i,j) \in E,\ t \in T \tag{14}$$

$$C \ge t\, s_{jkt} \qquad \forall j \in D,\ k \in K,\ t < \bar{T} \tag{15}$$

$$s_{jkt} = 0 \qquad \forall k \in K,\ j \in D \text{ with } \mathrm{idx}(j) < k,\ t < \bar{T} \tag{16}$$

mechanism the rule completes 62% of standard instances. The recovery mechanism is a timed override. It *starts* at the first decision point at which the fleet has been in partial deadlock for at least k_{stall} consecutive steps and no recovery is in progress. It *lasts* for h_{hold} decision points, that is, roughly $20\,h_{\text{hold}}$ steps, fewer when goals become invalid early; during these decision points the rule is evaluated as usual and then overridden as in step 6. It *ends* when the count e reaches zero, after which the plain rule resumes; if the deadlock persists it can start again at the next decision point. Two modes are implemented. In *lead swap* the current lead is sent back to the depot and the role of lead passes to another drone; on the m-th recovery of an episode it passes to the m-th farthest drone, so that repeated recoveries cycle through the fleet rather than swapping the same two drones. In *retreat* every drone except the lead returns towards the depot, which slackens the chain. Section 6.5 selects the mode and the two parameters on tuning seeds; lead swap with $k_{\text{stall}} = 120$ and $h_{\text{hold}} = 20$ is used in every comparison.

4.3. Search heuristic over visit orders

The rule decides myopically: at every decision point the lead goes to its nearest unserved demand point. The third method keeps everything else the rule does and replaces that choice by a search. A solution is a *priority list* of demand points per drone. At a decision point a drone's goal is the first demand point on its own list that is still unserved, or the depot if its list is exhausted or it is empty; relays and the deadlock recovery are assigned exactly as in Procedure 1, steps 3–6. The makespan of a solution is obtained by simulating it, so the objective needs no model of the dynamics and automatically respects the connectivity constraint.

The search is simulated annealing. The initial solution is read off a run of the rule, by recording which drone served which demand point in which order, so the search starts from the rule's behaviour and can only improve on it. A neighbour is produced by relocating one demand point to a random position in a random drone's list, or by swapping two demand points between lists; a candidate is accepted if it is not worse, and otherwise with the usual Metropolis probability under a geometric cooling schedule. Rollouts that exceed 2.5× the incumbent are cut short. We run three restarts of 1,000 iterations, so about 3,000 simulations per instance. Unlike the other two methods the search is offline: it needs the instance before the mission and spends minutes on it, which is why we report it as a quality reference rather than as a controller.

4.4. Hierarchical learned policy

The learned controller is hierarchical (Figure 3). The high level, a *manager*, acts only at decision points and outputs one option per drone; the low level is the shared straight-line flight of Section 4.1, so the manager's choices are held for 20 steps and the problem it solves is a semi-Markov decision process over options [60]. Only the manager is learned. Separating the levels has two consequences that the rest of the design relies on: the manager sees the fleet at a coarse time scale at which a formation can be held, and the low level, being a fixed rule, can be reasoned about and projected for connectivity without learning.

Set-based encoder. The manager encodes drone i from four kinds of entities, all distances divided by R. *Own state*, 10 features: cargo fraction; distance to the depot; hop depth divided

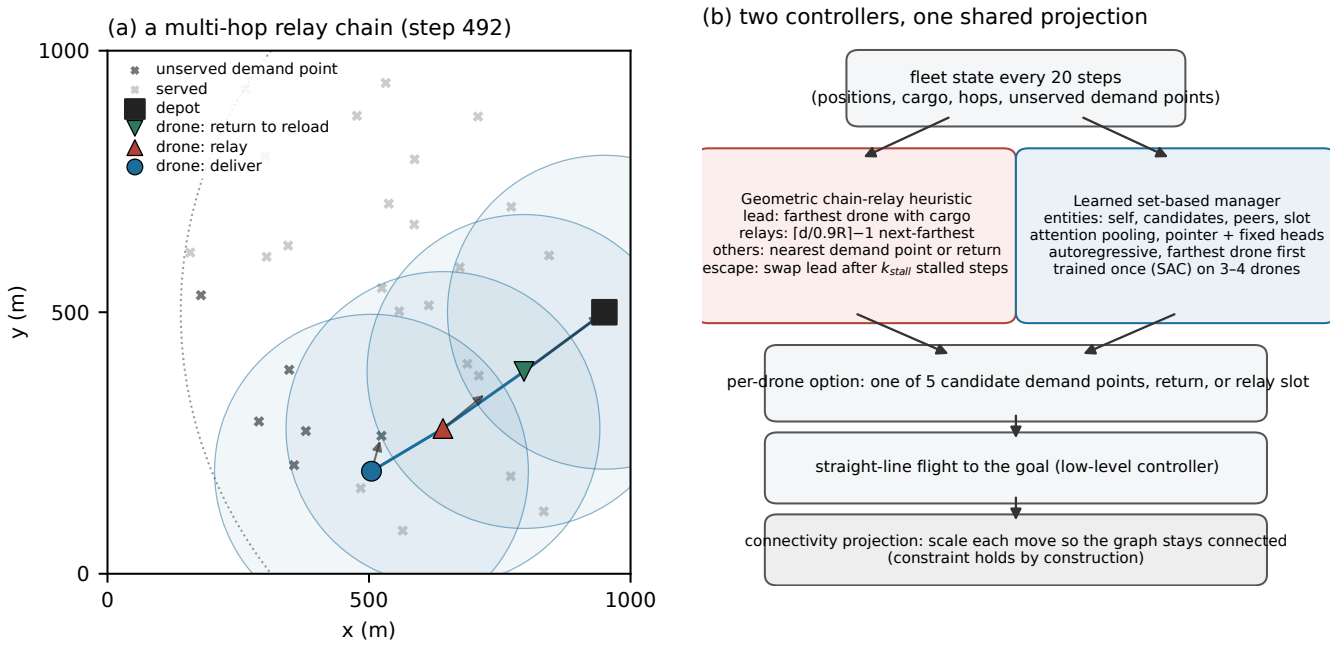


Figure 2: (a) An instance with three drones under the rule at step 499: the lead (blue circle) heads for a far demand point, one drone relays on the segment to the depot (red triangle), and one drone returns to the depot to reload (green triangle); shaded discs are communication ranges and the dotted circle is the reachable region $0.9NR$. (b) The two controllers emit the same per-drone options and share the straight-line low-level controller and the connectivity projection.

Procedure 1. Geometric chain-relay rule with deadlock recovery, run at every decision point. State kept between calls: remaining recovery length e (initially 0) and recovery count m (initially 0); both reset at the start of an episode. Parameters: $k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$, mode $\in$ {lead swap, retreat}.

1. **Recovery trigger.** If the fleet has been in partial deadlock for $\delta_t \geq k_{\text{stall}}$ consecutive steps and $e = 0$, set $e \leftarrow h_{\text{hold}}$ and $m \leftarrow m + 1$.
2. **Default assignment.** For each drone i: if it has no cargo, option $\leftarrow$ return; else if it has no unserved candidate, option $\leftarrow$ relay; else option $\leftarrow$ its nearest unserved demand point.
3. **Lead.** If no demand point is unserved, stop. Otherwise let $r_i = \|p_i - c\|$ and let the lead be the drone with cargo that maximises r_i (if no drone has cargo, the drone maximising r_i).
4. **Relay count.** Let $d = \max(r_{\text{lead}}, \|d_{\text{goal}} - c\|)$ where d_{goal} is the lead's nearest unserved demand point, and $n = \min(\max(1, \lceil d/(0.9R) \rceil) - 1,\ N - 1)$.
5. **Relay identity.** Sort the drones other than the lead by r_i in decreasing order and set the option of the first n of them to relay.
6. **Recovery manoeuvre** (only if $e > 0$; then $e \leftarrow e - 1$). *Lead swap*: sort all drones by r_i in decreasing order, let the new lead be the $\min(m, N-1)$-th drone of that order (0-based), set the old lead's option to return and, if the new lead has cargo and an unserved candidate, set its option to its nearest unserved demand point. *Retreat*: set the option of every drone except the lead to return.
7. Return the options; the environment converts them into goals and assigns relay slots.

by 8; fraction of demand points unserved; elapsed time divided by the budget; its realised-movement ratio (an exponential average of achieved over commanded displacement); whether it could advance one full step towards its goal alone without disconnecting the fleet; the fleet-wide realised-movement ratio; service timer; and whether it is a cut vertex of G_t. *Candidate demand points*, five entities of 6 features: distance; the two components of the unit direction vector; whether a teammate already targets it; its distance from the depot; and the hops its chain requires, divided by 8. *Peers*, $N - 1$ entities of 9 features: distance; the two components of the relative position vector; cargo fraction; hop depth divided by 8; and a four-way one-hot of the peer's current goal kind (deliver, return, hold, relay). *Relay slot*, 4 features: distance; the two components of the unit direction vector; and the number of slots in the current chain, divided by 8. Each entity type passes through a two-layer MLP of width 128. The drone's own embedding is the query of two multi-head attention layers (4 heads) that pool the peer set and the candidate set into one vector each; the four vectors are concatenated and fused by a further MLP into the drone's embedding h_i. A *pointer head* scores each candidate from $[h_i; e_{\text{cand}}]$ and a *fixed head* scores return and relay from h_i; the seven scores are masked and normalised by a softmax. Nothing in the network depends on N or M, which is why one set of weights runs on fleets it never saw.

Autoregressive decision. The N options are not sampled independently. Drones are ordered by distance from the depot, farthest first; the manager samples the first drone's option, writes its new role into the role one-hot of the peer features that the remaining drones see, re-encodes, samples the next drone, and so on (Procedure 2). A formation of one lead and k relays arises only by coincidence under independent sampling but can be composed deliberately under sequential decisions; the cost is N sequential network calls per decision point.

Training. The manager is trained with discrete soft actor-critic [18] on option transitions. A transition stores the fleet observation at a decision point, the N options, the option return $\sum_\tau \gamma^\tau r_\tau$ accumulated over the steps until the next decision point, the observation at that point, and the discount $\gamma^{\Delta t}$ with $\gamma = 0.997$

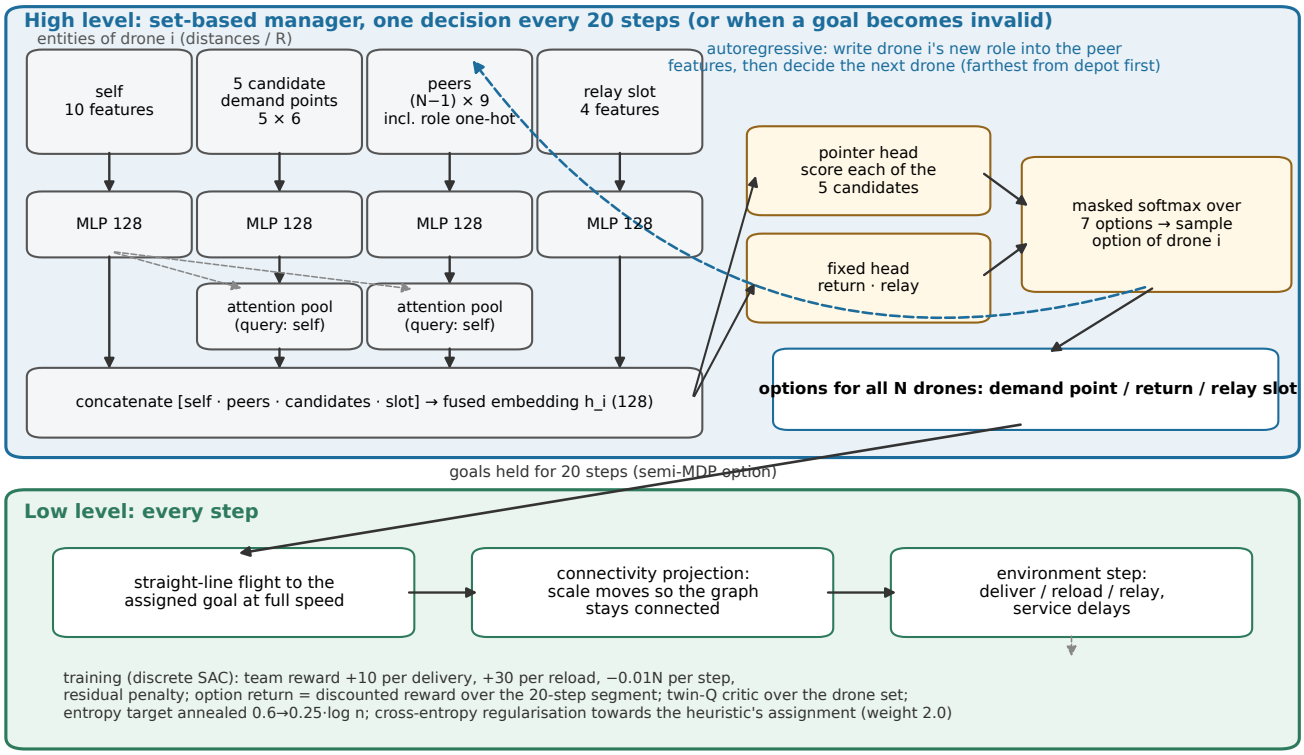


Figure 3: The hierarchical learned controller. High level: the set-based manager encodes each drone from four kinds of entities, pools the peer and candidate sets with attention queried by the drone's own embedding, and scores the five candidates with a pointer head and return/relay with a fixed head; drones are decided one at a time, farthest from the depot first, and each decision is written into the peer features seen by the next drone. Low level, every step: straight-line flight to the assigned goal, the connectivity projection, and the environment step. Options are held for 20 steps.

Procedure 2. Learned manager at a decision point (inference).

1. Build the entity observation of every drone from the fleet state; compute the validity mask of the seven options for every drone (Section 4.1).
2. Order the drones by distance from the depot, farthest first.
3. For each drone i in that order: encode all entities with the current peer roles, compute the seven masked logits of drone i, sample its option from the softmax (sampling, not argmax, is the deployed inference), and write the chosen role (deliver, return or relay) into the peer role one-hot of drone i as seen by the drones not yet decided.
4. Return the N options; the environment converts them into goals and assigns relay slots.

per step. The team reward is +10 per delivery, +30 per reload, $-0.01N$ per step and a residual penalty of $-0.1N$ times the fraction of demand points still unserved. A twin-Q critic with attention across the drone embeddings outputs per-drone action values whose mean over drones is regressed on the option return; the actor minimises the standard expectation form $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a) - Q(a)]$ per drone, with teacher forcing of the autoregressive order. Three elements of the training configuration matter and are ablated in Section 6.5. (i) The entropy target is annealed linearly from $0.6 \log n_{\text{valid}}$ to $0.25 \log n_{\text{valid}}$ over the first 400 episodes: a high target keeps the policy random enough to recovery deadlocks early in training, a low one lets it hold a multi-step formation later. (ii) A cross-entropy term with weight 2.0 pulls the policy towards the option the rule's assignment rule (Procedure 1, steps 2–5, without recovery) would take, wherever that option is valid. (iii) Every training episode samples $N \in \{3, 4\}$, $M \in [20, 80]$ and a random depot. Optimisation uses Adam at 3×10^{-4}, batches of 256 transitions from a replay buffer of 60,000, one gradient step every 4 environment steps after a warm-up of 400 transitions, and Polyak averaging of the target critic with $\tau = 0.005$. Checkpoints are scored every 50 episodes on 80 validation instances with a 3,000-step budget by $100f + 40(1 - \bar{t}/3000)$, where f is the completion rate and $\bar{t}$ the mean steps; training stops after eight evaluations without improvement or at 1,000 episodes. One run takes about 13 hours on a single RTX 5080 when four runs share the GPU.

5. Experimental Setup

Instances and configurations. Instances are generated from integer seeds; a seed, N, M and the depot mode determine the instance exactly, so two controllers can be run on identical instances. Evaluation seeds are disjoint from the seeds used for training, for checkpoint selection and for tuning the rule. We evaluate on the standard configuration ($N = 4$, $M = 50$, fixed depot) and on eight unseen configurations with random depots: $N/M \in \{3/20, 3/30, 4/80, 5/50, 6/80, 8/150, 10/300, 12/400\}$.

Baselines. Four learning-free baselines establish that the problem is not solved by naive assignment. All use the shared in-

terface of Section 4.1 and are evaluated at the same decision points. *Greedy, no relay* is step 2 of Procedure 1 alone: every drone with cargo flies to its nearest unserved demand point and every empty drone returns; nobody ever relays, so the chain is maintained only by the projection, which stalls the fleet. *One (two) fixed relays* assigns the first one (two) drones of the fleet to relay permanently and runs the greedy rule on the others; the relay count is fixed in advance instead of adapting to the lead's distance. *Geometric rule, no recovery* is Procedure 1 without step 6. All learning-free controllers are deterministic except for the random tie-breaking of the projection.

Reference model. Instances with $M \leq 5$ demand points are solved to proven optimality with the breadth-first search of Section 6.2 on the hull-pruned lattice; instances with $M \geq 6$ are solved with the space–time MILP in Gurobi 13 [66], four threads and one hour per instance. Every optimal or best-found plan is replayed in the continuous simulator.

Paired protocol. Averaging only the episodes each method completes favours the method that gives up on hard instances, and a single long stall moves a 40-run mean by over 100 steps. All comparisons between the two controllers are therefore *paired*: both run on the same instance, the difference is taken over instances both complete, and we report the median paired difference, the win–loss count and a Wilcoxon signed-rank p-value. Inference samples from the policy; deterministic argmax inference has the same median but a far heavier tail. The standard configuration uses 480 runs (60 seeds, 8 repetitions, two independent measurement runs), the 3/30 and 5/50 configurations 200 (40 seeds, 5 repetitions) and the remaining configurations 80 (40 seeds, 2 repetitions). Repetitions differ because the projection's tie breaking is random. We found 40 runs too few to support any claim about tails.

6. Results

6.1. RQ1: Completion under the hard constraint

Connectivity was never violated in any training episode or evaluation run, as the projection guarantees. Table 2 reports completion within the 10,000-step budget. The learned policy completes every instance in every configuration. The rule with recovery completes 98–100%; its misses concentrate where drones are plentiful, where a long chain that is not needed still gets stationed and stalls. Without the recovery mechanism the rule completes 61.7% of standard instances, and the naive baselines complete almost none: the greedy assignment pulls the fleet apart, and fixed relay counts are wrong for most instances because the number of relays a demand point needs varies with its distance.

Table 2: Completion within the 10,000-step budget. Learning-free baselines on 120 (standard) or 40 (unseen) runs; the two proposed controllers on the paired run counts of Section 5. p is Fisher's exact test (two-sided) of each method's completion count against the learned policy on the same configuration.

Method	4/50 std.	p	3/30	p	5/50	p
Greedy, no relay	0%	$< 10^{-4}$	0%	$< 10^{-4}$	25%	$< 10^{-4}$
One fixed relay	0%	$< 10^{-4}$	2.5%	$< 10^{-4}$	40%	$< 10^{-4}$
Two fixed relays	3.3%	$< 10^{-4}$	52.5%	$< 10^{-4}$	65%	$< 10^{-4}$
Rule, no recovery	61.7%	$< 10^{-4}$	95%	0.027	92.5%	0.004
Rule + recov. (prop.)	473/480	0.015	200/200	1	191/200	0.004
Learned (prop.)	**480/480**	—	**200/200**	—	**200/200**	—
	3/20		4/80		6/80	
Rule + recov. (prop.)	80/80	1	80/80	1	79/80	1
Learned (prop.)	**80/80**		**80/80**		**80/80**	
	8/150		10/300		12/400	
Rule + recov. (prop.)	80/80	1	80/80	1	80/80	1
Learned (prop.)	**80/80**		**80/80**		**80/80**	

6.2. RQ2: Distance to the exact optimum of the space–time model

Exact solutions. We fix the lattice at $h = 98$ m, the spacing derived in Section 3.1, and solve the space–time model on it to proven optimality by breadth-first search over its state space. A state is the multiset of the three drones' nodes together with the set of served demand points; the fleet is identical so states are kept up to permutation, and the search expands all node triples reachable in one period that satisfy the multi-hop connectivity test, period by period, until every demand point is served. Because every transition is exactly a feasible column of the MILP, the first period at which the full set appears is the optimal makespan of the model. The search does not carry cargo, which relaxes the capacity constraint; we recover the optimal plan and check it, and on all forty instances no drone delivers more than $Q = 5$ units, so the relaxation is tight and the values are the optima of the full model. Two checks confirm the implementation: on the instance Gurobi could certify for $M = 4$ the search returns the same optimum in one second against 1,242 (seed 902, ten periods), and on the three instances with $M = 6$ that the MILP certified within an hour the two agree exactly (130, 100 and 120 steps). Each instance takes 0–11 seconds, so we solve ten instances for each of $M = 3, 4, 5, 6$.

Running the controllers in the model's decision space. A controller that flies freely and a plan confined to the lattice are not playing the same game, so a difference between them is partly an artefact of the discretisation. We therefore run the controllers in the model's own decision space. A drone occupies a lattice node at every period boundary and may move only to a node in $A(i)$; connectivity is checked at period boundaries, as in the model; a service occupies a whole period, so a drone that stands on an unserved demand point with cargo at a boundary delivers and stays there for that period, which is exactly $s_{jkt} = 1$; relay slots are snapped to the nearest node. The low-level mover is itself close to optimal: node-to-node distances in periods are precomputed, and at each boundary the joint move minimising the total remaining periods to the drones' goals, subject to connectivity, is chosen, so the measured gap reflects goal selection and not a clumsy execution layer. The implementation is verified by replaying each optimal plan through the same machinery: on all forty instances the replay reproduces the model's optimum exactly, so the optimum is a valid lower bound for every controller in this space.

Result. Table 3 and Figure 4 give the gap of each method to the optimum. No run of any method beats the optimum. In the model's space the rule is 58.0% above it, the learned policy 55.8% and the search heuristic 20.8%; the search heuristic is better than both online controllers ($p < 10^{-4}$ in both cases, Wilcoxon over 39 instances) while the rule and the learned policy are statistically indistinguishable from each other ($p = 0.14$).

The three methods decompose the gap, because the search heuristic uses exactly the option structure of the rule — one chain, geometric relay slots, the same deadlock recovery — and optimises only the order in which demand points are visited. The 37 percentage points between the rule and the search heuristic are therefore the cost of choosing targets myopically, and the remaining 21 points are the cost of the option structure itself and of executing it on the lattice. The learned policy was meant to make exactly the choice the search heuristic optimises, and on these instances it does not: it sits with the rule. That is where the headroom is, and it is not where we expected it.

What the continuous measurement adds. The last two columns of Table 3 repeat the measurement with the controllers in the continuous space they are designed for, against the same lattice optima. There the rule is 9.7% and the learned policy 4.3% above the optimum, and on the smallest instances they are faster than it. Both statements are true and measure different things. The model's optimum is exact in its own space but is an upper bound on the continuous optimum, because its drones cover h rather than ℓ per period on axis-aligned legs and spend a whole period on each service; so the continuous figures are lower bounds on the true optimality gap, not estimates of it. The large difference between the two columns is not a property of learning: the rule is training-free and loses 48 percentage points in the move to the lattice, against 52 for the learned policy, so all but a few points of the difference are the cost of executing a controller designed for straight-line flight on a 98 m lattice with ten-step periods.

Cost. The search heuristic buys its 37 points with computation: a mean of 360 s per instance over 3,000 simulated rollouts, against 0.05–0.12 ms per decision for the rule and 2.7–7.4 ms for the learned policy (Table 11), and it needs the instance in advance. The exact search needs 0–11 s but only works while no reload is needed.

6.3. RQ3: Comparative performance across operating conditions

Fleet size divides the configuration space. Table 4 and Figure 5 give the paired comparison across the nine configurations. With three drones the rule is faster by 39 and 30 steps in paired median with 20 and 30 demand points. With four drones the two controllers are statistically indistinguishable: on the standard configuration the median paired difference over 480 runs is +2 steps with 234 wins against 239 ($p = 0.52$), and on 4/80 the learned policy is 38 steps faster. With five or more drones the learned policy is faster, by 62 steps at five drones growing to 178 at ten and 76 at twelve (70 wins against 10). The boundary therefore lies at four drones, and the advantage persists beyond the largest fleet we measured, although the monotone growth stops between ten and twelve (Section 6.4 returns to this point). An earlier measurement against the rule's pre-tuning recovery parameters gave the same picture (+40, +0, −54, −170 on 3/30, 4/50, 5/50 and 10/300), so the boundary does not depend on the tuning.

Figure 6 shows the full distributions on the three main configurations: with three drones the rule's curve lies to the left, with four the curves coincide, and with five the learned policy's curve lies to the left and reaches completion earlier.

Tails follow the same split. Where drones are scarce the learned policy's worst case is three to four times the rule's (maximum 5,395 versus 1,447 steps on 3/30); where they are plentiful it is lighter (1,101 versus 1,734 on 5/50, 1,365 versus 1,753 on 10/300). On the standard configuration over 480 runs against the pre-tuning rule the learned policy exceeded 1,500 steps five times and the rule fourteen times.

Stabilising the tail. That tail is a liability for a relief operation, where the worst case matters more than the median, so we tried to remove it from both sides. On the *training* side we trained four further runs under the proposed configuration: two with the deadlock-recovery override inside the rollout loop, so that the policy learns against the behaviour it will be deployed with, and two with the entropy target annealed to $0.10 \log n_{\text{valid}}$ instead of 0.25, to make the policy less random late in training. Neither worked. The lower entropy target doubled the spread on 3/30 (standard deviation 629 and 1,001 against 328 for the proposed policy, maxima of 6,283 and 9,049). Of the two runs with recovery in the loop, one matched the proposed policy on 3/30 (+36 against +30) and the other lost 110 steps of median, and evaluating the first of them *without* the override at inference restored the original spread (standard deviation 338, maximum 3,634), which shows that the training change did nothing and the override at inference did the work.

On the *inference* side, applying the rule's recovery step (Procedure 1, step 6) to the proposed policy's options whenever the fleet has been in partial deadlock for k_{stall} steps, with no retraining, removes the tail on every configuration (Table 5). The worst case falls from 4,614 to 1,210 steps on 3/30 and from 3,061 to 1,436 on the standard configuration, the standard deviation falls by a third to a half, and no run exceeds 1,500 steps where twelve did before. The median cost is confined to the configuration where the learned policy was already behind: +64 instead of +30 on 3/30, while the standard configuration improves slightly (−13 against +2, both insignificant) and 5/50 is unchanged (−54 against −62). We therefore report the plain policy as the proposal, because it is the faster one in the median, and recommend the override wherever the worst case is what has to be guaranteed.

Mechanism. The two controllers differ in one decision. On 3/30 the rule stations 1.39 relays on average and the learned policy 0.78; the learned fleet therefore spends 27% of drone-steps stalled by the projection against 8% for the rule. Overriding only the learned policy's relay assignment with the rule's,

Table 3: Optimum of the space–time model at $h = 98$ m versus the three proposed methods, three drones, ten instances per M. The optima are proven by the search of Section 6.2. The first group runs every method in the model's decision space, where the optimum is a valid lower bound; the second repeats rule and learned policy in the continuous space they are designed for, where the same optimum is an upper bound on the continuous optimum and the gaps consequently understate. Learned values are medians of 30 runs; the search heuristic is the best of three annealing restarts.

M	inst.	optimum	gap to the optimum, model's decision space				continuous space	
		(mean)	rule	heuristic	learned	learned+rec.	rule	learned
3	10	89	+50.4%	+24.0%	+50.0%	+52.0%	+2.3%	-3.6%
4	10	110	+52.1%	+16.4%	+51.5%	+49.6%	+5.4%	-0.3%
5	10	121	+62.8%	+23.8%	+58.8%	+59.4%	+14.0%	+8.7%
6	10	136	+67.6%	+19.1%	+72.5%	+59.1%	+17.8%	+10.7%
all	39	—	+58.0%	+20.8%	+55.8%	+55.1%	+9.7%	+4.3%

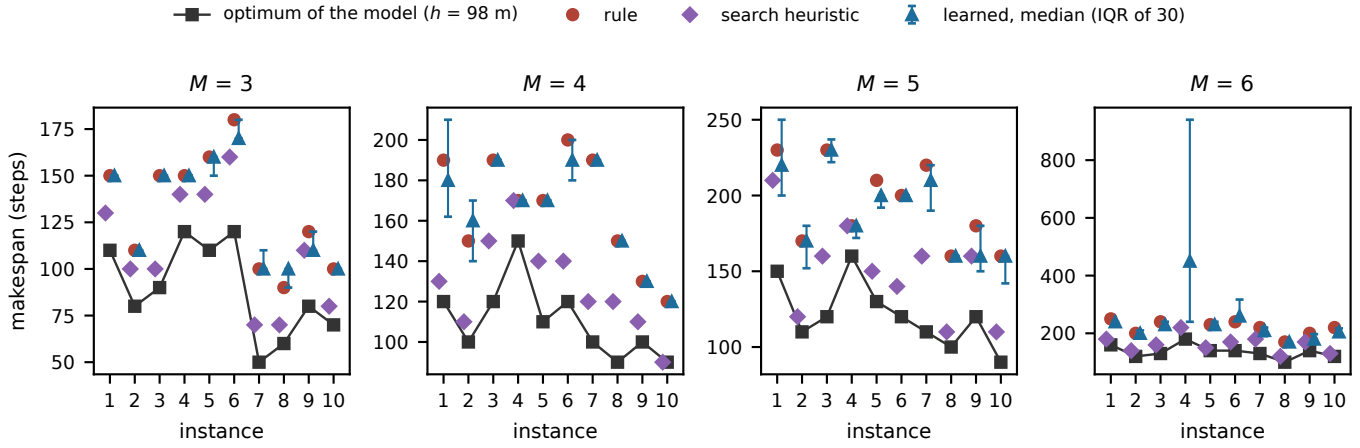


Figure 4: Per-instance makespan on the exactly solved instances with three drones, all methods run in the model's decision space: optimum of the space–time model at $h = 98$ m (square), rule (circle), search heuristic (diamond) and learned policy median with the interquartile range of 30 runs (triangle).

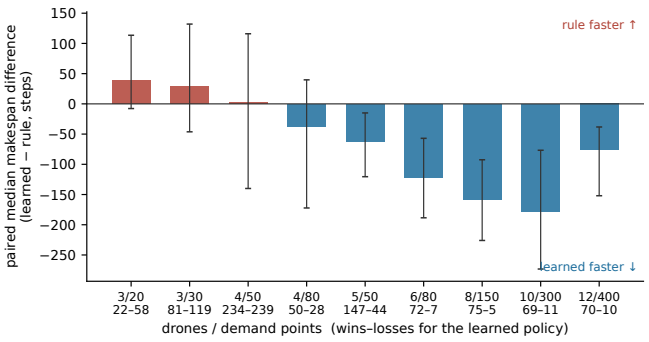


Figure 5: Paired median makespan difference (learned minus rule) by configuration, with the interquartile range of the paired differences as error bars and the learned policy's wins–losses under each label. Negative values favour the learned policy; the boundary lies at four drones.

at inference, recovers the full 44-step gap; matching only the *number* of relays recovers 14 steps of median but removes the tail (maximum from 4,904 to 1,083). The identity of the relaying drone drives the median and the count drives the tail. The information needed to make the rule's choice is present in the learned policy's observation: a two-layer classifier predicts the rule's relay assignment from the 41 observed features with 85% accuracy, and behaviour cloning of the full rule reaches 95% on relay decisions. When drones are plentiful the opposite happens: the rule keeps stationing a chain that the geometry no longer needs, while the learned policy exploits the slack and, with its sampled inference, recoveries the stalls that the deterministic rule repeats; this is also where the rule loses its few completions.

Why the relay-limited deficit resists policy-side fixes. Table 6 lists eleven interventions, each trained with four or two seeds under the same training configuration except for the stated change and evaluated with the same paired protocol, that attempt to make the learned policy relay like the rule when drones are scarce. The variants are, in the numbering of the table: (V1) the cross-entropy regularisation towards the rule is weighted three times higher for drones whose rule option is relay; (V2) each drone on the communication path of a delivering teammate receives an individual reward (2.0 or 6.0) at that delivery and the critic is trained on per-drone returns; (V3) checkpoint selection scores half of its validation instances on the 3/30 configuration instead of only on 4/50, eight seeds; (V4) as V2 but zero-sum, the delivering drone losing what its relays gain; (V5) the critic is conditioned on the teammates' chosen roles (a joint-action critic) with the teammates' next roles sampled from the actor in the target; (V6) the autoregressive order is reversed to decide the drone nearest the depot first; (V7) all goals are invalidated

Table 4: Paired comparison on identical instances. Positive differences mean the learned policy is slower than the rule. p from the Wilcoxon signed-rank test on both-completed pairs.

N/M	rollouts	complete h/l	median rule / learned	paired median	win–loss	p
3/20	80	80/80	364 / 420	+39	22–58	$< 10^{-4}$
3/30	200	200/200	572 / 618	+30	81–119	3.4e-04
4/50 (standard)	480	473/480	878 / 878	+2	234–239	0.521
4/80	80	80/80	1,120 / 1,036	−38	50–28	0.006
5/50	200	191/200	588 / 534	−62	147–44	$< 10^{-4}$
6/80	80	79/80	737 / 614	−122	72–7	$< 10^{-4}$
8/150	80	80/80	916 / 722	−158	75–5	$< 10^{-4}$
10/300	80	80/80	1,216 / 1,052	−178	69–11	$< 10^{-4}$
12/400	80	80/80	1,199 / 1,108	−76	70–10	$< 10^{-4}$

Table 5: Effect of the deadlock-recovery override applied to the proposed policy at inference, without retraining. The paired median, wins–losses and p are against the tuned rule on the same instances; the standard deviation, 90th percentile, maximum and the count above 1,500 steps describe the learned policy's own makespan distribution.

config.	inference	runs	paired median	wins–losses	p	s.d.	90th pct.	max	>1,500
3/30	proposed	200	+30	81–119	3.4e-04	328	837	4614	2
	+ recovery	200	+64	66–134	$< 10^{-4}$	149	858	1210	0
4/50	proposed	480	+2	234–239	0.521	208	1104	3061	7
	+ recovery	240	−13	125–110	0.249	132	1052	1436	0
5/50	proposed	200	−62	147–44	$< 10^{-4}$	125	712	1302	0
	+ recovery	200	−54	145–44	$< 10^{-4}$	131	734	1075	0

after 60 consecutive partial-deadlock steps so that the manager re-decides, in training and evaluation; (V8) the mean over per-drone action values in the critic is replaced by state-dependent monotone weights that sum to one with a floor of one half; (V9) each drone's option is stored as its own single-agent transition, chained inside a decision point with discount one, so that no decomposition of the team return is needed; (V10) the manager is initialised by behaviour cloning of the rule on 40,000 decisions (92.5% accuracy, 95% on relay decisions) and then fine-tuned, with and without a 50-episode critic warm-up; (V11) a per-step penalty proportional to the estimated arrival time of the bottleneck demand point, with the estimate divided by each drone's realised-movement ratio so that blocked drones count as slow. None improves the 3/30 median and several hurt the configurations where the learned policy wins. The pattern is informative. Adding reward for relaying turns the means into the end: the policy relays for its own sake and 5/50 reverses from −54 to +60. Every route to per-drone credit is blocked for a different reason: a joint-action critic inherits the variance of a sampled teammate action in its target; a monotonic mixing network either diverges when its weights are free or collapses to the mean when they are floored; a sequential single-agent reformulation removes the decomposition but leaves an undiscounted credit chain with no contraction. Initialising the policy by behaviour cloning of the rule, with 95% relay accuracy, is erased within 400 episodes: once the actor is updated, relay usage falls to the level of a randomly initialised policy within 50 episodes and stays there. A penalty on the bottleneck demand point's estimated arrival makes the policy reduce the *average* rather than the maximum. We conclude that, within this policy structure and objective, under-relaying is the learner's fixed point, not an artefact of credit, information or initialisation.

Sensitivity to the connectivity enforcement model. The projection of Section 4.1 scales a blocked displacement along its own direction, so a drone pulling on the chain effectively stops. We implemented an alternative that clamps the target position to the nearest point of the communication disc around the binding neighbour, which removes only the outward component and keeps the tangential one (a drone can still circle at constant range). Both controllers use the same projection in each comparison, and connectivity is still never violated. These measurements predate the rule tuning of Section 6.5 and use $k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$ for both projections, so they are internally consistent. Evaluating the same learned weights under the sliding projection without retraining reduces the 3/30 deficit from +40 to +21; retraining four seeds under it gives +15 to +48 (mean +28) and lowers the learned fleet's stall ratio from 0.27 to 0.17–0.22, with completion still 100%. The rule gains little with three drones (median 570 to 571) but gains more than the learned policy with four (866 to 812 against 879 to 850), so the standard-configuration equality becomes +26 in its favour. We therefore keep the radial projection as the primary model and report this as a sensitivity result: about half of the three-drone deficit is a property of how the constraint is enforced rather than of the learner.

Communication radius. Both controllers were run without retraining at $R = 250\,\text{m}$ and $R = 350\,\text{m}$ (Table 7, Figure 7); the learned policy normalises distances by R, and the instance generator rescales the reachable region $0.9NR$ with R, so both

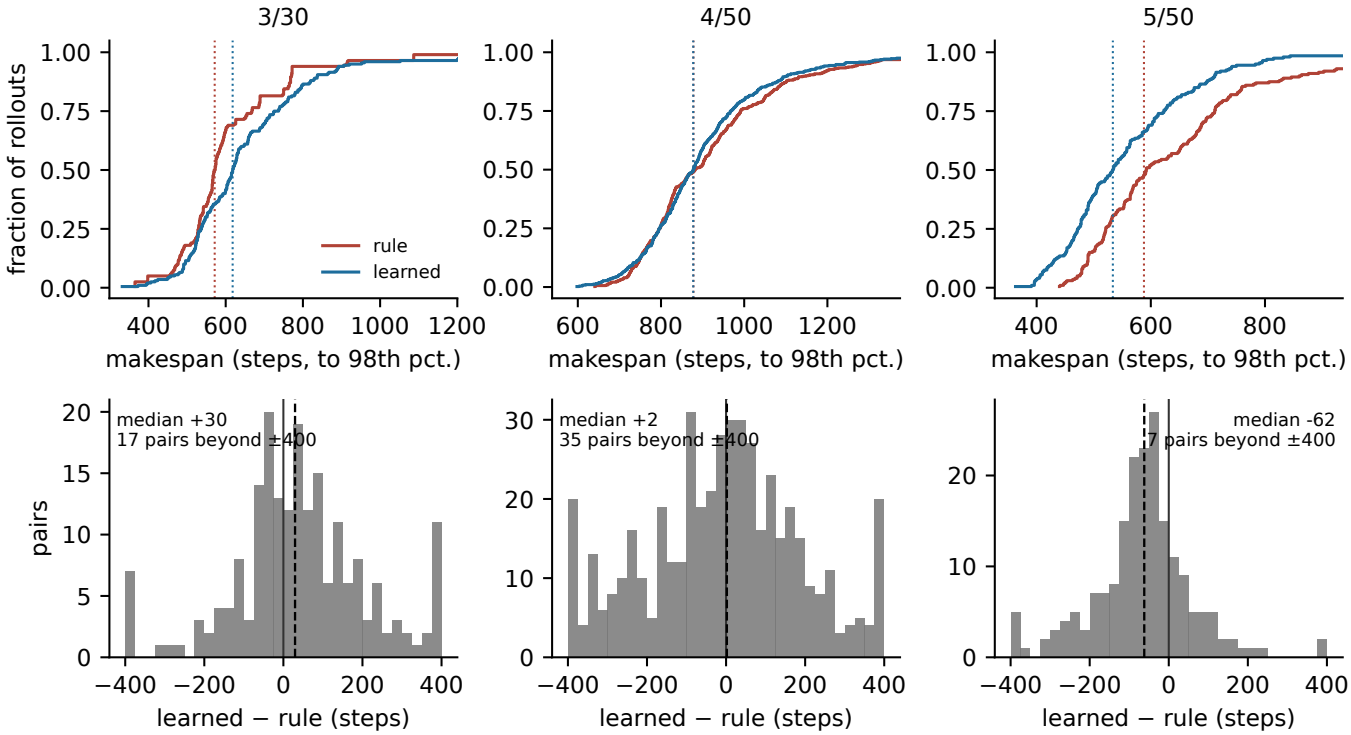


Figure 6: Top: cumulative distribution of makespan on 3/30, 4/50 and 5/50 (dotted lines mark medians; axes cut at the 98th percentile). Bottom: distribution of the paired difference on both-completed instances, clipped to ±400 steps with the number of clipped pairs noted.

controllers see the same instances at each radius. Completion transfers: the learned policy completes every instance at both radii, while the rule loses 18 of 200 instances on 5/50 at the shorter radius. Speed shows two effects. The first is the regime shift seen with fleet size: at $R = 250$ m a demand point needs more hops, the fleet is effectively scarcer, and every configuration moves towards the rule (+77 on 3/30, +68 on 4/50, and the 5/50 advantage shrinks to an insignificant −16 ($p = 0.33$)); at $R = 350$ m the fleet is effectively more plentiful and the learned policy is faster already with four drones (−40 on 4/50). The second is a training-radius effect: on 3/30 and 5/50 the learned policy's margin is best at the radius it was trained on and worse on both sides (+30 at 300 m against +77 and +52; −62 against −16 and −42), so the normalisation by R transfers completion but not the full speed. The regime variable is therefore the chain demand, the hops a demand point needs relative to the drones available, rather than the fleet size alone, and a deployment at a radius far from the training radius should retrain or at least re-evaluate the boundary.

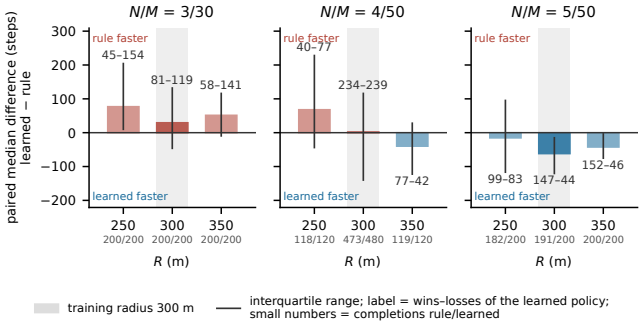


Figure 7: Transfer to other communication radii without retraining, one panel per configuration. Bars are the paired median difference (learned − rule, steps): a bar above zero (red) means the rule was faster, below zero (blue) the learned policy; whiskers are the interquartile range of the paired differences, the label at each bar is the learned policy's wins–losses, and the completions (rule/learned) are printed under the radius. The shaded column is the training radius. Reading left to right within a panel: a shorter radius moves every configuration towards the rule, and on 3/30 and 5/50 the learned policy's margin is best at the radius it was trained on.

6.4. RQ4: Upward transfer of the learned policy

The learned policy never saw more than four drones or more than 80 demand points in training, yet it completes every instance with 5 to 12 drones and 50 to 400 demand points and is faster than the rule on all of them. Two properties carry this. The set-based encoder has no parameter that depends on N or M, so larger fleets are simply larger sets; and training samples the fleet size and depot afresh every episode. Table 8 separates the two by running the ablation policy that was trained on the fixed 4/50 configuration alone (no configuration randomisation) on the same large-fleet instances without retraining. That policy also completes every instance from 5 to 10 drones, so completion generalises through the architecture alone; but it is slower than the randomised policy on every configuration (no significant difference from the rule at 5/50 against −62, and

Table 6: Interventions that did not remove the relay-limited deficit, numbered as in the text. Cells give the range over seeds of the paired median difference against the rule (positive: learned slower) and, in parentheses, the number of seeds whose difference is significant at $p < 0.05$ (Wilcoxon signed-rank) over the number of seeds trained. The reference row is the proposed policy against the tuned rule (Table 4); the variant rows were evaluated against the rule with its pre-tuning recovery parameters ($k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$), against which the proposed policy measured +40, −54 and +0, so differences of a few tens of steps between the reference row and a variant row are not meaningful.

#	Intervention (what changed)	3/30	5/50	4/50	Mechanism of failure
—	Proposed configuration (reference)	+30 (1/1)	−62 (1/1)	+0 (0/1)	—
V1	Relay-weighted rule regularisation (×3)	+18, +58 (1/2)	−29, +13 (0/2)	+13, +41 (1/2)	relay-specific regularisation harms
V2	Per-drone relay credit (2.0 / 6.0)	+36 to +82 (4/4)	+60 to +106 (4/4)	+39 to +131 (4/4)	relaying becomes the goal
V3	Checkpoint selection half on 3/30	+37 to +105 (8/8)	−58 to −22 (7/8)	−37 to +108 (3/8)	improves only the standard config.
V4	Zero-sum credit redistribution	+91, +126 (2/2)	+98, +172 (2/2)	+146, +180 (2/2)	same, larger
V5	Joint-action critic	+71 to +124 (4/4)	−46 to −16 (3/4)	+8 to +100 (2/4)	target variance from sampled teammates
V6	Nearest-first decision order	+46, +100 (2/2)	−43, −38 (2/2)	+4, +106 (1/2)	order is not the cause
V7	Re-decision after 60 stalled steps	+82, +95 (2/2)	−38, −34 (2/2)	+17, +55 (0/2)	irregular option lengths
V8	Monotone mixing critic (floor 0.5)	+54 to +230 (4/4)	−28 to +38 (2/4)	+55 to +258 (4/4)	divergence vs. credit on one axis
V9	Sequential single-agent credit	+81 to +213 (4/4)	+8 to +27 (2/4)	+62 to +170 (4/4)	no contraction within a decision
V10	Behaviour-cloning initialisation (± warm-up)	+6 to +69 (3/4)	−42 to −20 (3/4)	−16 to +40 (0/4)	erased within 400 episodes
V11	Bottleneck-arrival penalty (0.15 / 0.25)	+85 to +144 (4/4)	−32 to +13 (2/4)	+7 to +156 (3/4)	reduces the mean, not the maximum

Table 7: Transfer to other communication radii without retraining. Cells give completions (rule/learned) and the paired median difference with win–loss and p; $R = 300$ m is the training radius.

N/M	$R = 250$ m	$R = 300$ m	$R = 350$ m
3/30	200/200 / +77 (45–154, $< 10^{-4}$)	200/200 / +30 (81–119, 3.4e-04)	200/200 / +52 (58–141, $< 10^{-4}$)
4/50	118/120 / +68 (40–77, $< 10^{-4}$)	473/480 / +2 (234–239, 0.521)	119/120 / −40 (77–42, 1.1e-04)
5/50	182/200 / −16 (99–83, 0.327)	191/200 / −62 (147–44, $< 10^{-4}$)	200/200 / −42 (152–46, $< 10^{-4}$)

−51 to −174 against −122 to −178 beyond), so speed on large fleets is learned from the variety of small ones. The same ablation loses 114–296 steps on 3/30 (Table 10), so randomisation is what makes the small-fleet side usable at all. Both policies were trained with at most four drones; the hop-depth and slot features are normalised by a constant of eight, so fleets of ten and twelve drones also take the encoder outside its training range of feature values, and the policy still completes them.

Where the transfer weakens. We have not isolated the cause of the narrowing at 12/400 and offer three conjectures. First, both controllers maintain a single chain to one lead, so with twelve drones most of the fleet is delivering near the depot regardless of the controller and the decisions that differ between them concern a shrinking share of the work. Second, with 400 demand points a drone's five candidates are all close to it, so the choice among them matters less and the learned policy's advantage, which lies in choosing targets and relays jointly, has less to act on. Third, the manager's hop-depth and slot features are normalised by a constant of eight and exceed one for larger fleets, and twelve sequential autoregressive decisions compound sampling noise; both take the policy farther from its training distribution. Which of these dominates would require fleets beyond twelve drones to settle.

6.5. RQ5: Are the components necessary?

Rule. Table 9 grids $k_{\text{stall}} \in \{30, 60, 90, 120\}$, $h_{\text{hold}} \in \{20, 40, 80\}$ and the two recovery modes, plus no recovery, on 3/30, 4/50 and 5/50 with 40 tuning seeds each, and ranks settings by total completions first and the sum of median makespans second; this criterion was fixed before the measurement. The recovery mechanism is necessary: without it the rule completes 25–26 of 40 standard instances (two measurements). The mode matters more than the timing: every lead-swap setting completes 117–120 of 120 instances, while retreat settings complete 101–117 because slackening the chain does not change the assignment that caused the stall. Within the lead-swap family the differences are not statistically meaningful: in a second measurement of the grid, reported in the table together with a paired Wilcoxon test of every setting against the selected one on the same 40 instances, no setting differs from the selected one at $p < 0.05$ on any configuration (all $p \geq 0.09$), and one other setting (30/40) also completes all 120 with a median sum 13 steps lower, which is tie-breaking noise. The selected setting ($k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$) was fixed from the first measurement and is used throughout. The adaptive relay count is necessary as well: fixing one or two relays completes 0–65% (Table 2).

Learned policy. Table 10 removes one element at a time, retrains two seeds with the same budget, and runs each ablated policy on the same instances as the proposed policy (the first 120, 80 and 80 paired runs of Section 6.3); cells give the median of the per-instance makespan difference and a Wilcoxon signed-rank p, per seed. Rule regularisation and entropy-target annealing are necessary everywhere: removing either costs 52–189 steps on every configuration and seed at $p < 10^{-4}$. Configuration randomisation is necessary off the training distribution: removing it is neutral on the standard configuration (−19 and −34, $p \geq 0.12$), on which the ablated policy was trained, but costs 114–296 steps on 3/30 and 53–76 on 5/50. Autoregressive decisions help on every configuration by 32–88 steps ($p \leq 0.007$). The residual penalty is neutral: no seed on any configuration differs from the proposed policy at $p < 0.05$. A manager with a fixed-size MLP encoder instead of the set encoder, trained in an earlier version on the standard configuration only, completed 38% of standard instances and could not be run

Table 8: Transfer to fleets larger than any seen in training, without retraining. Cells give the learned policy's completions, the paired median difference against the rule (negative: learned faster), and in parentheses the learned policy's wins–losses and the Wilcoxon signed-rank p. The proposed policy is trained on $N \in \{3, 4\}$ with random depots; the comparison policy is the ablation trained on the fixed 4/50 configuration only (two seeds).

N/M	trained on $N \in \{3,4\}$, $M \in [20, 80]$	trained on 4/50 only, seed 1	seed 2
5/50	200/200 / −62 (147–44, $< 10^{-4}$)	80/80 / +7 (35–39, 0.840)	80/80 / +44 (28–50, 0.002)
6/80	80/80 / −122 (72–7, $< 10^{-4}$)	80/80 / −56 (57–20, $< 10^{-4}$)	80/80 / −51 (51–27, 0.010)
8/150	80/80 / −158 (75–5, $< 10^{-4}$)	80/80 / −129 (68–11, $< 10^{-4}$)	80/80 / −110 (60–20, $< 10^{-4}$)
10/300	80/80 / −178 (69–11, $< 10^{-4}$)	80/80 / −152 (69–11, $< 10^{-4}$)	80/80 / −174 (68–12, $< 10^{-4}$)

Table 9: Rule ablation on tuning seeds disjoint from the evaluation seeds (40 instances per configuration). Cells give completed instances, the median makespan over completed instances and, in parentheses, the p-value of a Wilcoxon signed-rank test of that setting against the selected setting (bold) on the instances both completed; the last column sums completions over the three configurations.

mode	k_{stall} / h_{hold}	3/30	4/50	5/50	completions
none	—	39/40 / 559 (0.228)	25/40 / 817 (0.258)	38/40 / 602 (0.093)	102/120
lead swap	30 / 20	40/40 / 557 (0.121)	39/40 / 853 (0.130)	40/40 / 625 (0.928)	119/120
lead swap	30 / 40	40/40 / 562 (0.673)	40/40 / 867 (0.775)	40/40 / 606 (0.247)	120/120
lead swap	30 / 80	40/40 / 567 (0.158)	39/40 / 844 (0.754)	39/40 / 614 (0.404)	118/120
lead swap	60 / 20	40/40 / 557 (0.599)	39/40 / 849 (0.780)	40/40 / 600 (0.791)	119/120
lead swap	60 / 40	40/40 / 554 (0.497)	39/40 / 852 (0.807)	40/40 / 625 (0.944)	119/120
lead swap	60 / 80	40/40 / 556 (0.359)	39/40 / 863 (0.788)	39/40 / 632 (0.675)	118/120
lead swap	90 / 20	40/40 / 566 (0.206)	39/40 / 871 (0.908)	40/40 / 599 (0.375)	119/120
lead swap	90 / 40	40/40 / 559 (0.473)	40/40 / 856 (0.674)	37/40 / 634 (0.844)	117/120
lead swap	90 / 80	40/40 / 563 (0.786)	39/40 / 858 (0.149)	39/40 / 629 (0.615)	118/120
lead swap	**120 / 20**	40/40 / 561	40/40 / 867	40/40 / 620	**120/120**
lead swap	120 / 40	40/40 / 557 (0.757)	39/40 / 878 (0.685)	40/40 / 624 (0.979)	119/120
lead swap	120 / 80	40/40 / 562 (0.687)	38/40 / 829 (0.546)	39/40 / 626 (0.850)	117/120
retreat	30 / 20	37/40 / 559 (0.350)	35/40 / 836 (0.185)	40/40 / 632 (0.528)	112/120
retreat	30 / 40	39/40 / 556 (0.079)	29/40 / 832 (0.284)	39/40 / 621 (0.157)	107/120
retreat	30 / 80	38/40 / 561 (0.754)	26/40 / 804 (0.443)	37/40 / 584 (0.368)	101/120
retreat	60 / 20	39/40 / 556 (0.866)	38/40 / 847 (0.215)	38/40 / 603 (0.759)	115/120
retreat	60 / 40	39/40 / 559 (0.875)	39/40 / 823 (0.485)	39/40 / 624 (0.459)	117/120
retreat	60 / 80	39/40 / 563 (0.991)	26/40 / 795 (0.236)	40/40 / 635 (0.490)	105/120
retreat	90 / 20	39/40 / 566 (0.512)	38/40 / 822 (0.058)	40/40 / 642 (0.773)	117/120
retreat	90 / 40	39/40 / 566 (0.180)	39/40 / 852 (0.634)	39/40 / 618 (0.615)	117/120
retreat	90 / 80	39/40 / 562 (0.665)	36/40 / 830 (0.578)	37/40 / 598 (0.354)	112/120
retreat	120 / 20	39/40 / 556 (0.645)	37/40 / 863 (0.327)	38/40 / 614 (0.460)	114/120
retreat	120 / 40	39/40 / 560 (0.757)	37/40 / 831 (0.451)	38/40 / 628 (0.602)	114/120
retreat	120 / 80	39/40 / 561 (0.191)	38/40 / 854 (0.815)	37/40 / 627 (0.308)	114/120

on other fleet sizes at all.

6.6. Inference cost

Table 11 gives the latency of one manager decision on a CPU. The rule costs 0.05–0.12 ms; the learned policy 1.3–2.5 ms without autoregression and 2.7–7.4 ms with it, growing linearly with the fleet because each drone is a sequential network call. Over a 900-step episode the learned policy's cumulative decision time is 0.16 s. The exact search takes 0–11 seconds on instances with three to six demand points at $h = 98$ m, and the MILP for larger instances uses its full hour.

7. Discussion

Deployment. The two controllers are complementary and the boundary is sharp, so a fleet operator can choose by fleet size: the rule with three drones, the learned policy with five or more, where it is both faster and more reliable; with four the two are statistically indistinguishable in speed and the learned policy completes more. Both decide in milliseconds and both keep the fleet connected by construction.

What the negative results mean. Eleven interventions that change how the learner is credited, informed, initialised or rewarded leave the relay-limited deficit in place, and a change to the environment's enforcement model removes half of it. The deficit is therefore best understood as an interaction between the learner's fixed point under this objective and the geometry of the constraint, not as a shortcoming that more training or a better critic would fix. For a practitioner the actionable consequence is the one above: pick the controller by regime.

Limitations. The learned policy was trained at a single communication radius; it transfers to radii of 250 and 350 m in completion but its speed ranking against the rule shifts with the radius

Table 10: Ablation of the learned policy. Each cell gives, for two independently trained seeds, the median per-instance makespan difference between the ablated policy and the proposed policy on the same instances (positive: the removed element was needed) and the Wilcoxon signed-rank p.

Removed component	4/50 (120 runs)	3/30 (80)	5/50 (80)
Rule regularisation	+121 ($< 10^{-4}$) / +189 ($< 10^{-4}$)	+171 ($< 10^{-4}$) / +160 ($< 10^{-4}$)	+52 ($< 10^{-4}$) / +152 ($< 10^{-4}$)
Configuration randomisation	−19 (0.260) / −34 (0.119)	+114 ($< 10^{-4}$) / +296 ($< 10^{-4}$)	+53 (2.7e-04) / +76 ($< 10^{-4}$)
Entropy target annealing	+136 ($< 10^{-4}$) / +138 ($< 10^{-4}$)	+138 ($< 10^{-4}$) / +130 ($< 10^{-4}$)	+58 ($< 10^{-4}$) / +76 ($< 10^{-4}$)
Autoregressive decisions	+32 (0.007) / +84 ($< 10^{-4}$)	+88 (3.2e-04) / +56 (0.002)	+42 (2.7e-04) / +60 (9.1e-04)
Residual penalty	−4 (0.746) / −32 (0.204)	+16 (0.625) / −28 (0.291)	+24 (0.120) / +12 (0.374)

Table 11: Latency of one manager decision (milliseconds, CPU) and the cumulative decision time of a 900-step episode.

Controller	$N = 3$	$N = 4$	$N = 6$	$N = 8$	900 steps
Geometric rule + recovery	0.05	0.08	0.10	0.12	0.003 s
Learned, independent decisions	1.27	1.52	1.94	2.45	0.069 s
Learned, autoregressive (proposed)	2.70	3.52	5.32	7.35	0.158 s

(Section 6.3), and radii outside this range were not tested. The environment is planar, ignores wind, battery and collisions, and uses a disc communication model; real links are not discs. The exact optimum is that of a discretised model at a single lattice spacing: the 98 m lattice and the ten-step period leave a quantisation that refining to 49 m shrinks by 7–9% but that we cannot afford on every instance, so the measured gaps understate the true ones, and beyond six demand points only solver incumbents with a remaining optimality gap are available. The rule's novelty lies in the recovery mechanism and the adaptive relay count rather than in the chain geometry itself.

Methodological notes. Three practices mattered for the conclusions. Completed-episode averages favour the controller that gives up, so comparisons must be paired. Forty runs cannot support claims about tails; several of our own interim conclusions reversed when the sample grew. And a pre-check that uses a different metric from the final judgement will pass interventions that the final judgement rejects.

8. Conclusion

We proposed two controllers for connectivity-constrained drone delivery, a geometric chain-relay rule with deadlock recovery and a fleet-size-agnostic learned policy, both acting through a projection that keeps the fleet connected at every step. Under a paired protocol across fleets of three to twelve drones the learned policy completes every instance and is faster by 38 to 178 steps once more than four drones are available, although it was trained on three and four, while the rule is faster by 30–39 steps with three. The relay-limited deficit is one decision, which drone relays; it resists eleven policy-side interventions and halves when the enforcement model is changed. Where an exact optimum of the space–time model is available, the learned policy is 4.3% above it on average and the rule 9.7%, and both decide three to six orders of magnitude faster than the exact search, whose reach ends at about six demand points. The natural next step is a policy whose output is a team assignment rather than a set of per-drone options, so that the relay identity becomes a coordinate the critic can value directly.

Acknowledgements

[To be added.]

Data and code availability

Code, trained weights, instance generators and the paired-comparison records are available at `https://github.com/koptimizer/relay_drone_network`.

References

[1] S. Chowdhury, A. Emelogu, M. Marufuzzaman, S. G. Nurre, L. Bian, Drones for disaster response and relief operations: A continuous approximation model, International Journal of Production Economics 188 (2017) 167–184.

[2] B. Rabta, C. Wankmüller, G. Reiner, A drone fleet model for last-mile distribution in disaster relief operations, International Journal of Disaster Risk Reduction 28 (2018) 107–112.

[3] M. Erdelj, E. Natalizio, K. R. Chowdhury, I. F. Akyildiz, Help from the sky: Leveraging UAVs for disaster management, IEEE Pervasive Computing 16 (1) (2017) 24–32.

[4] S. J. Kim, G. J. Lim, J. Cho, M. J. Côté, Drone-aided healthcare services for patients with chronic diseases in rural areas, Journal of Intelligent & Robotic Systems 88 (2017) 163–180.

[5] Z. Ghelichi, M. Gentili, P. B. Mirchandani, Logistics for a fleet of drones for medical item delivery: A case study for Louisville, KY, Computers & Operations Research 135 (2021) 105443.

[6] C. C. Murray, A. G. Chu, The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery, Transportation Research Part C: Emerging Technologies 54 (2015) 86–109. `doi:10.1016/j.trc.2015.03.005`.

[7] A. Otto, N. Agatz, J. Campbell, B. Golden, E. Pesch, Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey, Networks 72 (4) (2018) 411–458.

[8] G. Macrina, L. Di Puglia Pugliese, F. Guerriero, G. Laporte, Drone-aided routing: A literature review, Transportation Research Part C: Emerging Technologies 120 (2020) 102762.

[9] S. H. Chung, B. Sah, J. Lee, Optimization for drone and drone-truck combined operations: A review of the state of the art and future directions, Computers & Operations Research 123 (2020) 105004.

[10] Y. Zeng, R. Zhang, T. J. Lim, Throughput maximization for UAV-enabled mobile relaying systems, IEEE Transactions on Communications 64 (12) (2016) 4983–4996. `doi:10.1109/TCOMM.2016.2611512`.

[11] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, M. Debbah, A tutorial on UAVs for wireless networks: Applications, challenges, and open problems, IEEE Communications Surveys & Tutorials 21 (3) (2019) 2334–2360.

[12] O. Burdakov, P. Doherty, K. Holmberg, J. Kvarnström, P.-M. Olsson, Relay positioning for unmanned aerial vehicle surveillance, The International Journal of Robotics Research 29 (8) (2010) 1069–1087.

[13] M. M. Zavlanos, M. B. Egerstedt, G. J. Pappas, Graph-theoretic connectivity control of mobile robot networks, Proceedings of the IEEE 99 (9) (2011) 1525–1540.

[14] C. Dixon, E. W. Frew, Maintaining optimal communication chains in robotic sensor networks using mobility control, Mobile Networks and Applications 14 (3) (2009) 281–291.

[15] L. R. Ford, D. R. Fulkerson, Flows in Networks, Princeton University Press, Princeton, NJ, 1962.

[16] M. Skutella, An introduction to network flows over time, in: Research Trends in Combinatorial Optimization, Springer, Berlin, 2009, pp. 451–482.

[17] R. Roberti, M. Ruthmair, Exact methods for the traveling salesman problem with drone, Transportation Science 55 (2) (2021) 315–335.

[18] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 1861–1870.

[19] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: Proceedings of the 7th International Conference on Learning Representations, 2019.

[20] J. Park, C. Kwon, J. Park, Learn to solve the min-max multiple traveling salesmen problem with reinforcement learning, in: Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems, 2023, pp. 878–886.

[21] G. B. Dantzig, J. H. Ramser, The truck dispatching problem, Management Science 6 (1) (1959) 80–91.

[22] P. Toth, D. Vigo (Eds.), Vehicle Routing: Problems, Methods, and Applications, 2nd Edition, SIAM, Philadelphia, PA, 2014.

[23] P. Kitjacharoenchai, M. Ventresca, M. Moshref-Javadi, S. Lee, J. M. A. Tanchoco, P. A. Brunese, Multiple traveling salesman problem with drones: Mathematical model and heuristic approach, Computers & Industrial Engineering 129 (2019) 14–30.

[24] C. C. Murray, R. Raj, The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones, Transportation Research Part C: Emerging Technologies 110 (2020) 368–398. doi:10.1016/j.trc.2019.11.003.

[25] N. Agatz, P. Bouman, M. Schmidt, Optimization approaches for the traveling salesman problem with drone, Transportation Science 52 (4) (2018) 965–981. doi:10.1287/trsc.2017.0791.

[26] Q. M. Ha, Y. Deville, Q. D. Pham, M. H. Hà, On the min-cost traveling salesman problem with drone, Transportation Research Part C: Emerging Technologies 86 (2018) 597–621.

[27] M. Dell'Amico, R. Montemanni, S. Novellani, Drone-assisted deliveries: New formulations for the flying sidekick traveling salesman problem, Optimization Letters 15 (2021) 1617–1648.

[28] S. Poikonen, X. Wang, B. Golden, The vehicle routing problem with drones: Extended models and connections, Networks 70 (1) (2017) 34–43.

[29] D. Sacramento, D. Pisinger, S. Ropke, An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones, Transportation Research Part C: Emerging Technologies 102 (2019) 289–315.

[30] D. Schermer, M. Moeini, O. Wendt, A matheuristic for the vehicle routing problem with drones and its variants, Transportation Research Part C: Emerging Technologies 106 (2019) 166–204.

[31] K. Dorling, J. Heinrichs, G. G. Messier, S. Magierowski, Vehicle routing problems for drone delivery, IEEE Transactions on Systems, Man, and Cybernetics: Systems 47 (1) (2017) 70–85.

[32] V. Pillac, M. Gendreau, C. Guéret, A. L. Medaglia, A review of dynamic vehicle routing problems, European Journal of Operational Research 225 (1) (2013) 1–11.

[33] İ. Bekmezci, O. K. Sahingoz, Ş. Temel, Flying ad-hoc networks (FANETs): A survey, Ad Hoc Networks 11 (3) (2013) 1254–1270.

[34] L. Gupta, R. Jain, G. Vaszkun, Survey of important issues in UAV communication networks, IEEE Communications Surveys & Tutorials 18 (2) (2016) 1123–1152.

[35] O. Tekdas, W. Yang, V. Isler, Robotic routers: Algorithms and implementation, The International Journal of Robotics Research 29 (1) (2010) 110–126.

[36] E. Yanmaz, Positioning aerial relays to maintain connectivity during drone team missions, Ad Hoc Networks 128 (2022) 102800.

[37] M. M. Zavlanos, G. J. Pappas, Potential fields for maintaining connectivity of mobile networks, IEEE Transactions on Robotics 23 (4) (2007) 812–816.

[38] M. Li, Y. Jie, Y. Kong, H. Cheng, Decentralized global connectivity maintenance for multi-robot navigation: A reinforcement learning approach, arXiv:2109.08536 (2021).

[39] R. Ding, J. Chen, W. Wu, J. Liu, F. Gao, X. Shen, Packet routing in dynamic multi-hop UAV relay network: A multi-agent learning approach, IEEE Transactions on Vehicular Technology 71 (9) (2022) 10059–10072.

[40] MUTTO: Multi-agent reinforcement learning for joint trajectory and transmit power optimisation of relay and mission UAVs, Vehicular Communications (2024).
URL https://www.sciencedirect.com/science/article/abs/pii/S1570870524002130

[41] MAEN: Multi-agent aerial relay networking, Proceedings of ACM MobiCom (2024). doi:10.1145/3636534.3694730.

[42] J. De Hoog, S. Cameron, A. Visser, Role-based autonomous multi-robot exploration, in: Proceedings of the International Conference on Advanced Cognitive Technologies and Applications, 2009.

[43] K. Cesare, R. Skeele, S.-H. Yoo, Y. Zhang, G. Hollinger, Multi-UAV exploration with limited communication and battery, in: Proceedings of the IEEE International Conference on Robotics and Automation, 2015, pp. 2230–2235. doi:10.1109/ICRA.2015.7139494.

[44] N. Boland, M. Hewitt, L. Marshall, M. Savelsbergh, The continuous-time service network design problem, Operations Research 65 (5) (2017) 1303–1321.

[45] A. Richards, J. P. How, Aircraft trajectory planning with collision avoidance using mixed integer linear programming, in: Proceedings of the American Control Conference, 2002, pp. 1936–1941.

[46] H. D. Sherali, J. C. Smith, Improving discrete model representations via symmetry considerations, Management Science 47 (10) (2001) 1396–1407.

[47] F. Margot, Symmetry in integer linear programming, in: 50 Years of Integer Programming 1958–2008, Springer, Berlin, 2010, pp. 647–686.

[48] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems 28, 2015, pp. 2692–2700.

[49] M. Nazari, A. Oroojlooy, L. V. Snyder, M. Takáč, Reinforcement learning for solving the vehicle routing problem, in: Advances in Neural Information Processing Systems 31, 2018, pp. 9839–9849.

[50] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421.

[51] N. Mazyavkina, S. Sviridov, S. Ivanov, E. Burnaev, Reinforcement learning for combinatorial optimization: A survey, Computers & Operations Research 134 (2021) 105400.

[52] Y. Guo, Z. Ren, C. Wang, iMTSP: Solving min-max multiple traveling salesman problem with imperative learning, arXiv:2405.00285 (2024).

[53] Y. Zhu, H. Xu, A. A. Malikopoulos, N. Geroliminis, Dual-mission drone fleets: Delivery with secondary sensing tasks, arXiv:2604.02471 (2026).

[54] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: Advances in Neural Information Processing Systems 30, 2017, pp. 6379–6390.

[55] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 2018, pp. 2974–2982.

[56] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems, 2018, pp. 2085–2087.

[57] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, S. Whiteson, QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 4295–4304.

[58] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu, The surprising effectiveness of PPO in cooperative multi-agent games, arXiv:2103.01955 (2022).

[59] A. Oroojlooy, D. Hajinezhad, A review of cooperative multi-agent deep reinforcement learning, Applied Intelligence 53 (2023) 13677–13722.
[60] R. S. Sutton, D. Precup, S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning, Artificial Intelligence 112 (1–2) (1999) 181–211.
[61] P.-L. Bacon, J. Harb, D. Precup, The option-critic architecture, in: Proceedings of the 31st AAAI Conference on Artificial Intelligence, 2017, pp. 1726–1734.
[62] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, K. Kavukcuoglu, FeUdal networks for hierarchical reinforcement learning, in: Proceedings of the 34th International Conference on Machine Learning, 2017, pp. 3540–3549.
[63] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, in: Proceedings of the International FLAIRS Conference, Vol. 35, 2022.
[64] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, Y. Tassa, Safe exploration in continuous action spaces, arXiv:1801.08757 (2018).
[65] A. Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: Proceedings of the 16th International Conference on Machine Learning, 1999, pp. 278–287.
[66] Gurobi Optimization, LLC, Gurobi optimizer reference manual, version 13, `https://www.gurobi.com` (2026).